Feature Engineering in Time Series Forecasting

Subrata Karmaker
M.Sc in Mathematics, Technische Universität Chemnitz, Germany
skarmaker.tuc@gmail.com

November 2025

Abstract

Feature engineering is a crucial element in Time Series Forecasting. Features can be designed to capture how patterns, trends, and seasonality evolve over time. The following sections introduce practical techniques inspired by calendar information and Fourier terms: lag features, rolling and seasonal window aggregations, exponentially weighted moving averages, and temporal embeddings. It also shows common issues arising, such as data leakage, and clarifies why well-defined forecast horizons are so important. Finally, an experimental illustration using German electricity load data demonstrates the importance of Feature Engineering in forecasting performance on real-world time series. The aim is to give an intuitive and accessible overview that helps practitioners turn time series into richer, more informative inputs for their forecasting models.

Contents

1 Introduction

By analyzing historical data and identifying patterns, one can make informed predictions about future trends and outcomes. Feature engineering is a major determinant in this process since it involves the selection and transformation of the most relevant input variables, which improve the accuracy of the time series forecasting models [2, 7].

The purpose of this paper is to discuss why feature engineering matters for time series and to illustrate several techniques that we have found useful in practice. We outline common feature families, show how they can be combined, and highlight where extra care is needed during design and evaluation. This paper builds on the conceptual framework presented in Chapter 6 of *Modern Time Series Forecasting with Python* by M. Joseph (2022) and extends it through reinterpreted explanations and an independent empirical study using German electricity load data [7].

2 The Significance of Feature Engineering

The traditional machine learning model for time series forecasting, for example, ARIMA, doesn't need feature engineering to understand time, because it is already built into the model [1, 7]. However, though ARIMA models are effective for capturing linear relationships and simple temporal dependencies, it may struggle with more intricate patterns. It can also become computationally expensive. In contrast, since the baseline machine learning regression models don't have the feature "time" incorporated into them, we need to create good features to embed the temporal aspect of the problem. This method can handle large datasets and capture complex non-linear relationships [3, 7].

In the feature engineering process, new features are generated based on past observations. This process relies on a continuous representation along the time axis. So, operationally, it is better to combine the training, validation, and test sets. This is when a problem occurs [7].

3 A Cautionary Aspect: Data Leakage

Data leakage occurs when information from the future (beyond the point where predictions are supposed to be made) is inadvertently used during the model training process. When this happens, it usually leads to overly optimistic outcomes during the model-building phase, followed by the unpleasant surprise of poor results after the prediction model is implemented and tested on new data [4, 7].

Data leakage can occur in various forms, but the two primary types are:

Train-Test Contamination: Leakage occurs when information from the test (or validation) dataset seeps into the training dataset, leading to an overly optimistic performance on the test set but poor generalization to unseen data.

Target Leakage: Target leakage occurs when features directly related to the target variable are provided as training data. These features may not be available in a real-world prediction, causing unrealistically high model accuracy.

One common practice to prevent data leakage is to split the time series data into training and validation sets so that the validation set comes after the training set. This ensures that the model cannot access future information when making predictions on the validation set. Furthermore, in the case of feature engineering using lag or rolling window aggregations, it is crucial to use only the past data to derive these features without incorporating any information from the future [2, 7].

We will have to be very cautious about each of the features to ensure we are not using any data that will not be available during the prediction. The following diagram will help us remember and internalize this concept:

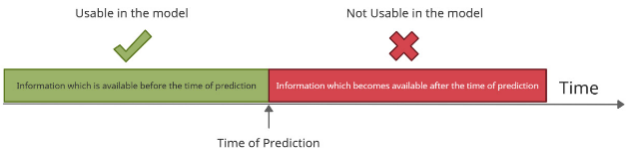

Figure 1: Usable and not-usable information to avoid data leakage[7]

4 Setting a Forecast Horizon

A forecast horizon is the number of time steps into the future we want to forecast at any point in time. A well-defined forecast horizon ensures useful insights for specific time periods and affects overall forecasting success. If the evaluation of the model includes data from the forecast horizon or beyond, it can give a misleadingly optimistic assessment of the model's performance. The model may appear more accurate than it is because it has seen data that it shouldn't have during training, which leads to data leakage [2, 7].

5 Common Feature Engineering Techniques

5.1 Time Delay Embedding

Time delay embedding involves creating a new representation of time series data by including past observations as features. There are a few more ways to capture past observations using this concept. Let's take a look [5, 7]:

5.1.1 Lags or Backshift

Let us consider a scenario about predicting the stock price of a company. So, the previous day's stock price is important to make a prediction, right? In other words, the value at time T is greatly affected by the value at time $T-1$. The

past values are known as lags, so $T-1$ is lag 1, $T-2$ is lag 2, and so on. The following diagram is a representation of this:

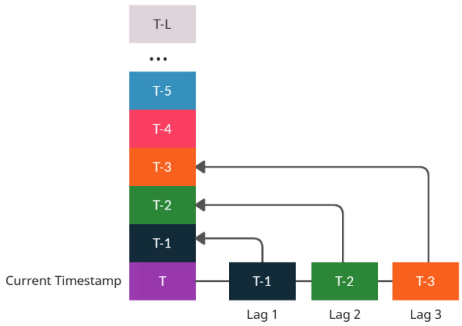

Figure 2: Lag features[7]

By adding observations from "a" timesteps before (denoted as "lag a"), we can generate multiple lags [7].

Let's consider a scenario where the historical sales data for a specific product in the retail chain are as follows:

Week	Sales (units)
1	100
2	120
3	150
4	180

Table 1: Weekly sales

In this scenario, incorporating a lag of one week would transform the data as follows:

Week	Sales (lag 1)	Sales (units)
2	100	120
3	120	150
4	150	180

Table 2: Weekly sales with lag features

By including past sales data, the forecasting model can account for previous dependencies and time-based patterns, resulting in more precise forecasts for upcoming periods.

5.1.2 Rolling Window Aggregations

Rolling window aggregations is a widely used technique that is built upon calculating summary statistics, such as the mean or standard deviation, over a

sliding window of previous values. This method can help capture trends and patterns in the data [2, 7].

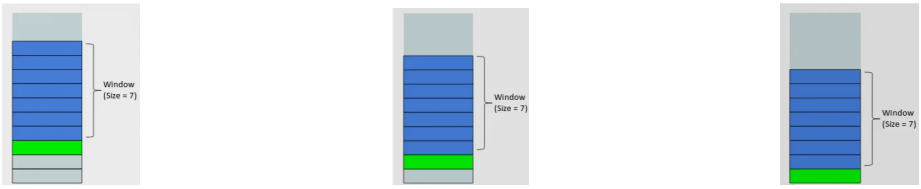

Figure 3: Rolling window concept[7]

Since this looks like a window that is sliding with every next point, the features generated using this method are called the "rolling window" features. Then we take the aggregated statistics, such as the mean, of the values in the window and use it as a feature for timestep t. A smaller window captures short-term fluctuations, while a larger window captures longer-term trends [7].

Let's take a look at the following diagram to understand it better.

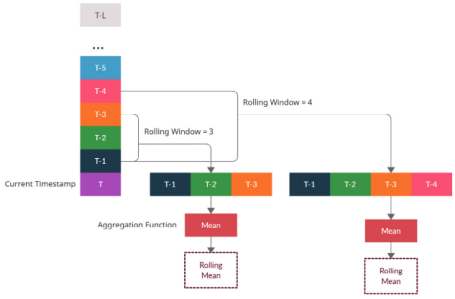

Figure 4: Rolling window over time

To illustrate the concept with a real-life example, let's consider a dataset that represents daily temperature readings in a specific location:

Date	Temperature (°C)
2021-01-01	20
2021-01-02	22
2021-01-03	18
2021-01-04	25
2021-01-05	23

Table 3: Daily temperature

Applying a rolling window aggregation, such as the 3-day moving average, to the temperature data would result in the following:

Date	Temperature (°C)	3-Day Moving Average
2021-01-01	20	NaN
2021-01-02	22	NaN
2021-01-03	18	20
2021-01-04	25	21.67
2021-01-05	23	22

Table 4: Rolling window aggregations

The 3-day moving average provides a smoothed representation of the temperature trends, enabling the model to capture the underlying patterns in the data more effectively [7].

5.1.3 Seasonal Rolling Window Aggregations

Similar to standard rolling window aggregations, seasonal rolling window aggregations involve calculating summary statistics over a specified window of previous observations within the same seasonal period [7].

Let's continue with the example of daily temperature readings. To capture seasonal temperature variations, we can apply a seasonal rolling window aggregation to calculate the 7-day moving average for each calendar month. This would enable the model to identify monthly temperature trends and variations specific to each season, such as summer or winter.

The following diagram will make this clearer:

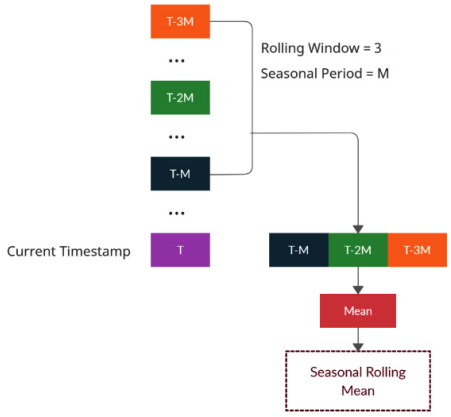

Figure 5: Seasonal rolling window aggregations

Here M is the seasonality period, which is the number of timesteps after which we expect the seasonality pattern to repeat.

5.1.4 Exponentially Weighted Moving Averages (EWMA)

In the rolling window aggregation operation, we compute the average in each window equally. In EWMA, the moving average is designed in such a way that the older observations are given lower weights. The weights decay exponentially as the data point gets older. The EWMA's simple mathematical formulation is described below [7]:

$$EWMA_t = \alpha \times y_t + (1 - \alpha) \times EWMA_{t-1}$$

The only decision a user of the EWMA must make is the parameter α. The parameter determines how important the current observation is in the calculation of the EWMA. The higher the value of α, the more closely the EWMA tracks the recent time series.

The EWMA is a recursive function, which means that the current observation is calculated using the previous observation.

The weights of each term would be:

$$w\_\{t - k\} \ = \ \alpha \times (1 - \alpha)\hat{}k$$

Here k is the number of timesteps behind t.

The EWMA can be calculated for a given day range, like 20-day EWMA or 200-day EWMA. To compute the moving average, we first need to find the corresponding alpha, which is given by the formula below:

$$\alpha = \frac{2}{1 + span}$$

Span is the number of time periods for which the moving average is calculated. For example, a 15-day moving average's alpha is given by $2/(15 + 1)$, which means alpha is 0.125. Since alpha is between 0 and 1, the weight becomes smaller as k becomes larger.

The following charts show how the weights decay for different values of α:

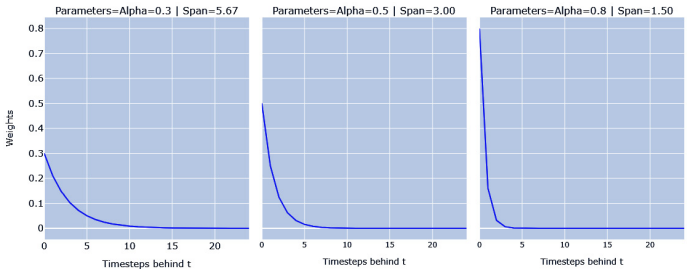

Figure 6: Exponential weight decay for different values of α[7]

Intuitively, we can think of EWMA as an average of the entire history of the time series. Still, with parameters such as α and **span**, we can make different periods of history more representative of the average [1, 7].

5.2 Temporal Embedding

Temporal embedding for feature engineering in time series analysis involves transforming time-related information into a format suitable for machine learning models, allowing the model to capture and leverage temporal patterns and dependencies for accurate forecasting [6, 7].

In time series forecasting, two aspects of time are important: the passage of time and the periodicity of time. Let's look at a few features that can help us capture these aspects in an ML model.

5.2.1 Calendar Features

The calendar features capture the periodicity of time and help the ML model capture seasonality well. Instead of relying solely on the raw chronological order of timestamps, calendar features provide the model with explicit information about the temporal context, which can be especially relevant for time series data that exhibits regular patterns based on the calendar [2, 7].

For example, in a time series forecasting task for retail sales, a calendar feature could provide information on whether a timestamp corresponds to a weekday, weekend, holiday, or regular day. This helps the model capture and use the natural patterns and seasonal variations of specific days or months.

5.2.2 Time Elapsed

Time elapsed features capture the passage of time in an ML model. In the context of retail sales forecasting, the time elapsed between consecutive sales events can be used as a feature to account for seasonality, cyclic patterns, and other time-based variations. This temporal information can be leveraged to identify and capture the evolving trends and patterns in the data, allowing the forecasting model to make more accurate predictions for future sales periods [7].

5.2.3 Fourier Terms

When dealing with high-frequency data or various seasonal patterns, typical ML models can become complicated and computationally expensive. Using Fourier terms can help in this situation. This technique can be applied to model and forecast time series data with multiple seasonal patterns, such as daily, weekly, or monthly seasonality [2, 7].

The basic idea behind the Fourier series is that any periodic function can be broken down into a sum of sine and cosine waves with different frequencies, amplitudes, and phases. These waves are known as Fourier components.

The sine-cosine form of the Fourier series is as follows:

$$s_{N(x)} = \frac{a_0}{2} + \Sigma_{n=1}^{N}\left(a_n \cdot cos\left(\frac{2\pi}{P} \cdot n \cdot x\right) + b_n \cdot sin\left(\frac{2\pi}{P} \cdot n \cdot x\right)\right)$$

Here, S_N is the N-term approximation of the signal S. And P is the maximum length of the cycle. We can create these cosine and sine functions as features to represent the seasonal cycle. Now if we are representing the seasonal cycle in months, P in this case would be 12 and x would be $1, 2, \ldots, 12$. Therefore, for each x, we can calculate the cosine and sine terms and add them as features to the ML model. The following diagram shows the difference in representations between the months on an ordinal scale and as a Fourier series:

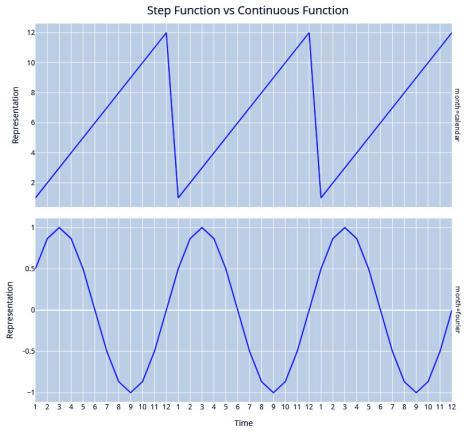

Figure 7: Month as an ordinal step function (top) versus Fourier terms (bottom)

The preceding diagram shows just a single Fourier term. The complexity of the seasonal pattern will determine how many Fourier terms to use [2, 7].

6 Experimental Illustration: German Electricity Load Forecasting

To demonstrate the practical effect of feature engineering techniques in real-world forecasting, an experiment was conducted using hourly electricity load data for Germany. The dataset was obtained from the *Open Power System Data* (OPSD) project, which compiles publicly available time series of electricity load, generation, and prices for European countries based on ENTSO-E and other official sources [8]. The German load time series covers multiple years with hourly resolution and exhibits clear daily and weekly seasonal patterns.

6.1 Experimental Setup

The experiment is based on the hourly "time series" dataset provided by the *Open Power System Data* (OPSD) project [8]. From the full multi-country file, we extract the German load series `DE_load_actual_entsoe_transparency` and its corresponding timestamp `utc_timestamp`. Rows with missing load values are removed. The resulting data frame has two columns: `Date` (in UTC) and `Load` (in megawatts).

Starting from this cleaned series, we construct two sets of predictors that correspond to the baseline and feature-engineered models:

- **Lag features:** we create lagged versions of the load at 1, 24, and 168 hours, denoted `lag1`, `lag24`, and `lag168`. These capture hourly, daily, and weekly dependencies respectively.
- **Rolling statistic:** we compute a 24-hour rolling mean of the load, `roll24`, to smooth local fluctuations and provide a short-term trend estimate.
- **Fourier terms:** we add sinusoidal features to represent daily and weekly seasonality. Using the time index $t = 0, 1, \ldots,$ we compute

$$\sin\left(\frac{2\pi t}{24}\right), \quad \cos\left(\frac{2\pi t}{24}\right), \quad \sin\left(\frac{2\pi t}{24 \cdot 7}\right), \quad \cos\left(\frac{2\pi t}{24 \cdot 7}\right),$$

  stored as `sin_daily`, `cos_daily`, `sin_weekly`, and `cos_weekly`.

Because lagging and rolling operations introduce missing values at the start of the series, we drop all rows with any missing predictor. This ensures that the final design matrix is complete for model fitting.

The data are then split chronologically: the first 80% of the observations form the training set and the remaining 20% form the test set. This respects the temporal order and avoids data leakage. The target variable y is always the current load `Load`.

We fit two linear regression models using `scikit-learn`. The baseline model uses only `lag1` as predictor. The feature-engineered model uses the full set of predictors

$$\{\texttt{lag1}, \texttt{lag24}, \texttt{lag168}, \texttt{roll24}, \texttt{sin\_daily}, \texttt{cos\_daily}, \texttt{sin\_weekly}, \texttt{cos\_weekly}\}.$$

Both models are trained on the training portion and evaluated on the held-out test portion. We report Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) in megawatts.

For visualization, we plot a short test segment (the first 200 hours of the test set) to compare the actual load with the predictions from both models, and we examine the corresponding residuals. This provides a qualitative complement to the aggregate error metrics and helps illustrate how feature engineering changes the shape and magnitude of the prediction errors.

6.2 Results and Discussion

Table 5 summarizes the performance comparison between the baseline and feature-engineered models. The results correspond to those obtained using the reproducible Python implementation in the Appendix.

Table 5: Model performance comparison on German hourly electricity load data

Model	**MAE (MW)**	**RMSE (MW)**
Baseline (lag1 only)	1884.68	2443.89
Feature-engineered linear regression	1231.59	1637.02

The baseline model, which uses only the previous hour's value, provides a naive but often surprisingly strong benchmark in short-horizon forecasting. However, once lag features for 24 and 168 hours, rolling statistics, and Fourier seasonality terms are added, the feature-engineered model substantially outperforms the baseline, reducing both MAE and RMSE.

These results confirm that even simple models benefit greatly from informative time-derived features. Lagged variables capture autocorrelation, rolling averages stabilize short-term noise, and Fourier terms model recurring cyclical behavior such as daily and weekly load patterns. This improvement demonstrates the practical power of feature engineering in enhancing the predictive accuracy and robustness of time series models [7, 2].

6.3 Visualization of Model Behavior

While the numerical results clearly show a reduction in error metrics, visual distinctions between the models can appear subtle because both capture the large-scale seasonal behavior of electricity load. To better illustrate the improvement, two visualizations were created.

Figure 8 shows a short segment of the actual and predicted load curves for both the baseline and feature-engineered models. The feature-engineered forecast tracks the true load curve more closely, especially around peaks and troughs.

Figure 9 then compares the residual errors (differences between actual and predicted values) for both models. The residuals from the feature-engineered model fluctuate more tightly around zero, indicating smaller and less systematic prediction errors.

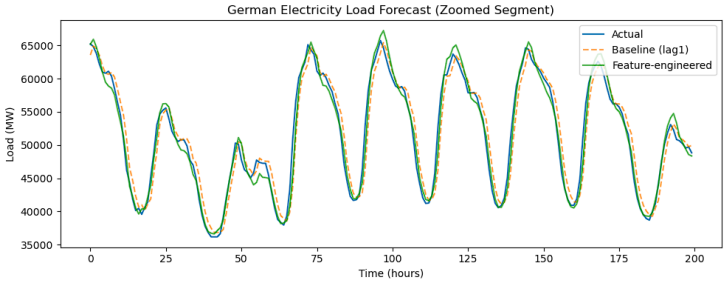

Figure 8: Short forecast segment comparing actual, baseline, and feature-engineered predictions.

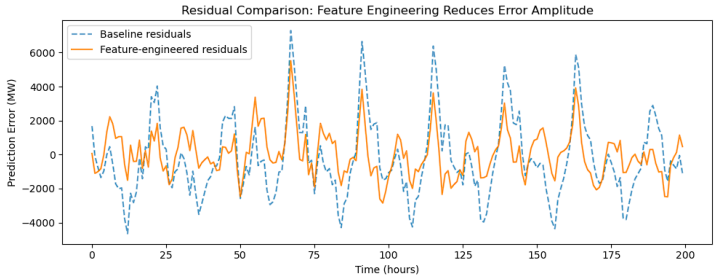

Figure 9: Residual comparison between baseline and feature-engineered models on German load data.

7 Conclusion

In this article, we discussed several practical feature engineering techniques that can be used to work with time series data. Using these techniques, we can convert any time series problem into a supervised learning problem and build regression models. The experimental illustration with German electricity load data clearly shows that well-engineered features can significantly improve forecasting performance, validating the theoretical insights discussed throughout the paper.

Reproducibility Note

All code and data used in this study are publicly available. The electricity load dataset for Germany can be obtained from the *Open Power System Data* repository [8]. The Python implementation, included in the Appendix, uses open-source libraries (`pandas`, `numpy`, `matplotlib`, and `scikit-learn`) and can be executed without modification on any standard Python 3 environment. The exact results in Table 5 can be reproduced by running the provided script, which

automatically downloads the dataset from the OPSD website and performs all preprocessing, feature engineering, and model training steps.

Related Work

FTime series forecasting feature engineering is a popular topic in the classical and modern literature on machine learning. Initial research by Box and Jenkins introduced the basis of statistical modeling via ARIMA and expo-nential smoothing, in which temporal dependence is directly represented in model parameters [1]. Nevertheless, with the rise in popularity of machine learning and ensemble models, researchers have noted that these models necessitated explicit time-dependent features which represented time dependencies [3, 5, 7].

Recent studies have brought about the focus on integrating engineered features and auto-mated learning. Lai et al. suggested hybrid deep learning models based on Long Short-Term Memory (LSTM) and convolutional networks that have extra lag and calendar features as optimally handcrafted [6]. On the same note, Bandara et al. and Hyndman et al. have shown that the old feature engineer-ing methods like Fourier terms, moving averages and lagged features are still useful in increasing interpretability and enhancing accuracy even to neural and ensemble based models [2]. Joseph [7] gives a practical, end-to-end workflow of these feature engineering methods using modern Python-based workflows in forecasting.

Altogether, these works point to the fact that feature engineering is an important part of time series forecasting and it brings features of interpretability and robustness that data-driven-only approaches may not.

Acknowledgements

The author would like to express sincere gratitude to the author and publishers of *Modern Time Series Forecasting with Python* by Joseph [7]. Chapter 6 of this book, in particular, provided the inspiration, structure, and many of the examples that form the basis of this article.

References

[1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[2] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021. Available online: `https://otexts.com/fpp3/`

[3] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.

[4] M. Kuhn and K. Johnson, *Applied Predictive Modeling*. New York, NY, USA: Springer, 2013.

[5] J. Brownlee, *Introduction to Time Series Forecasting with Python*. Machine Learning Mastery, 2017.

[6] G. Lai, W. Chang, Y. Yang, and H. Liu, "Modeling long- and short-term temporal patterns with deep neural networks," in *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2018, pp. 95–104.

[7] M. Joseph, *Modern Time Series Forecasting with Python: Explore Industry-Ready Time Series Forecasting Using Modern Machine Learning and Deep Learning*. Packt Publishing, 2022.

[8] Open Power System Data, "Time series," version 2020-10-06, 2020. Available online: `https://doi.org/10.25832/time_series/2020-10-06`

A Python Implementation for the German Electricity Load Forecasting Experiment

The following Python script reproduces the feature engineering and modeling steps used in the experimental illustration on German electricity load forecasting. It shows how a simple baseline model and a feature-engineered model can be constructed and evaluated.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error,
    mean_squared_error
import matplotlib.pyplot as plt

# Load dataset from Open Power System Data (OPSD)
url = "https://data.open-power-system-data.org/time_series
    /2020-10-06/time_series_60min_singleindex.csv"
df = pd.read_csv(url, parse_dates=['utc_timestamp'])

# Select German load column and drop missing values
germany_load = df[['utc_timestamp', '
    DE_load_actual_entsoe_transparency']].dropna()
germany_load = germany_load.rename(columns={
    'utc_timestamp': 'Date',
    'DE_load_actual_entsoe_transparency': 'Load'
})

# Create lag features: 1 hour, 24 hours (daily), 168 hours (
    weekly)
```

```
germany_load['lag1'] = germany_load['Load'].shift(1)
germany_load['lag24'] = germany_load['Load'].shift(24)
germany_load['lag168'] = germany_load['Load'].shift(168)

# Rolling mean over the past 24 hours
germany_load['roll24'] = germany_load['Load'].rolling(window
    =24).mean()

# Fourier terms for daily and weekly seasonality
t = np.arange(len(germany_load))
germany_load['sin_daily'] = np.sin(2 * np.pi * t / 24)
germany_load['cos_daily'] = np.cos(2 * np.pi * t / 24)
germany_load['sin_weekly'] = np.sin(2 * np.pi * t / (24 * 7)
    )
germany_load['cos_weekly'] = np.cos(2 * np.pi * t / (24 * 7)
    )

# Drop rows with missing values created by shifting/rolling
germany_load = germany_load.dropna()

# Train-test split (80% train, 20% test)
split_index = int(len(germany_load) * 0.8)
train = germany_load.iloc[:split_index]
test = germany_load.iloc[split_index:]

y_train = train['Load']
y_test = test['Load']

# 1) Baseline model (lag1 only)
X_train_base = train[['lag1']]
X_test_base = test[['lag1']]

baseline_model = LinearRegression()
baseline_model.fit(X_train_base, y_train)
y_pred_base = baseline_model.predict(X_test_base)

mae_base = mean_absolute_error(y_test, y_pred_base)
rmse_base = mean_squared_error(y_test, y_pred_base, squared=
    False)

print("Baseline model (lag1 only)")
print("MAE:", round(mae_base, 2))
print("RMSE:", round(rmse_base, 2))
print()

# 2) Feature-engineered model (lags + rolling + Fourier)
feature_cols = [
    'lag1', 'lag24', 'lag168',
    'roll24',
    'sin_daily', 'cos_daily',
```

```
    'sin_weekly', 'cos_weekly'
]

X_train_feat = train[feature_cols]
X_test_feat = test[feature_cols]

feat_model = LinearRegression()
feat_model.fit(X_train_feat, y_train)
y_pred_feat = feat_model.predict(X_test_feat)

mae_feat = mean_absolute_error(y_test, y_pred_feat)
rmse_feat = mean_squared_error(y_test, y_pred_feat, squared=
    False)

print("Feature-engineered linear regression")
print("MAE:", round(mae_feat, 2))
print("RMSE:", round(rmse_feat, 2))

# Visualization: zoomed forecasts and residual comparison
plt.figure(figsize=(10, 4))
plt.plot(y_test.values[:200], label='Actual', linewidth=1.5)
plt.plot(y_pred_base[:200], label='Baseline (lag1)',
    linestyle='--', alpha=0.7)
plt.plot(y_pred_feat[:200], label='Feature-engineered',
    linestyle='-', alpha=0.8)
plt.title('German Electricity Load Forecast (Zoomed Segment)
    ')
plt.xlabel('Time (hours)')
plt.ylabel('Load (MW)')
plt.legend()
plt.tight_layout()
plt.show()

# Residual comparison
res_base = y_test.values - y_pred_base
res_feat = y_test.values - y_pred_feat

plt.figure(figsize=(10, 4))
plt.plot(res_base[:200], label='Baseline residuals',
    linestyle='--', alpha=0.7)
plt.plot(res_feat[:200], label='Feature-engineered residuals
    ', linestyle='-', alpha=0.8)
plt.title('Residual Comparison: Feature Engineering Reduces
    Error Amplitude')
plt.xlabel('Time (hours)')
plt.ylabel('Prediction Error (MW)')
plt.legend()
plt.tight_layout()
plt.savefig('residuals_plot.png', dpi=300)
plt.show()
```

Listing 1: Python code for baseline and feature-engineered forecasting using German electricity load data